

Information Retrieval and Text Mining

Index and Query Optimizations

Prof dr ir Jan C. Scholtes
Edition 2025-2026

Agenda for Today

- Phrase Queries and Positional Indices
- Dealing with Spelling Errors
- Faster Postings mergers (skip pointers)
- Heap and Zipf laws
- Index Construction
- Optimizing the index

Learning Goals Today

- Understand Phrase Queries and Positional Indices
- Learn how to deal with Spelling Errors
- Understand basic index data structures
- Understand Heap & Zipf laws
- Create faster indexes:
 - Faster Postings mergers (skip pointers)
 - Compress indexes
- Understand Incremental and distributed indexing
- Understand Hardware Considerations



Phrase queries and positional indexes



Maastricht University

Phrase queries

- Want to be able to answer queries such as “***Maastricht University***” – as a phrase
- Thus, the sentence “*I went to University in Maastricht*” is not a match.
 - The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works
 - Many more queries are *implicit phrase queries*
- For this, it no longer suffices to store only *<term : docs>* entries

A first attempt: Biword indexes

- Index every consecutive pair of terms in the text as a phrase
- For example, the text “Friends, Romans, Countrymen” would generate the biwords
 - *friends romans*
 - *romans countrymen*
- Each of these biwords is now a dictionary term
- Two-word phrase query-processing is now immediate.

Issues for biword indexes

- False positives, as noted before
- Index blowup due to bigger dictionary
 - Infeasible for more than biwords, big even for them
- Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy

Solution 2: Positional indexes

- In the postings, store for each ***term*** the position(s) in which tokens of it appear:

<***term***, number of docs containing ***term***;

doc1: position1, position2 ... ;

doc2: position1, position2 ... ;

etc.>

Positional index example

- For phrase queries, we use a merge algorithm recursively at the document level
- But we now need to deal with more than just equality

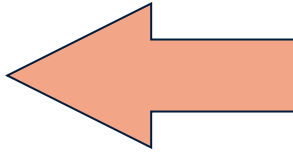
<*be*: 993427;

1: 7, 18, 33, 72, 86, 231;

2: 3, 149;

4: 17, 191, 291, 430, 434;

5: 363, 367, ...>



Which of docs *1,2,4,5*
could contain “*to be*
or not to be”?

Processing a phrase query

- Extract inverted index entries for each distinct term: ***to, be, or, not.***
- Merge their *doc:position* lists to enumerate all positions with “***to be or not to be***”.
 - ***to:***
 - 2:1,17,74,222,551; 4:8,16,190,429,433; 7:13,23,191; ...
 - ***be:***
 - 1:17,19; 4:17,191,291,430,434; 5:14,19,101; ...
- Same general method for proximity searches

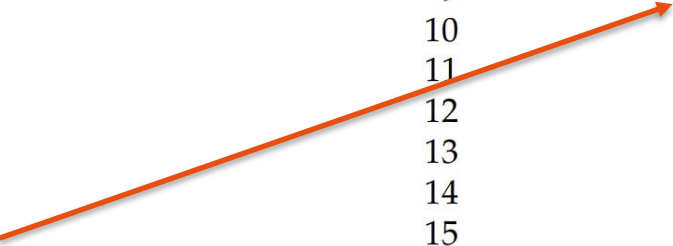
Proximity queries

- LIMIT! /3 STATUTE /3 FEDERAL /2 TORT
 - Again, here, / k means “within k words of”.
- Clearly, positional indexes can be used for such queries; biword indexes cannot.
- Exercise: Adapt the linear merge of postings to handle proximity queries. Can you make it work for any value of k ?
 - This is a little tricky to do correctly and efficiently
 - See Figure 2.12 of IIR (next slide)
 - There's likely to be a problem on it!

Limitation to value of k

But, in real-world applications, k is typically the length of a few sentences or a small paragraph.

```
POSITIONALINTERSECT( $p_1, p_2, k$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8              do while  $pp_2 \neq \text{NIL}$ 
9                  do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                     then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                     else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                         then break
13                      $pp_2 \leftarrow \text{next}(pp_2)$ 
14                     while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                         do  $\text{DELETE}(l[0])$ 
16                     for each  $ps \in l$ 
17                         do  $\text{ADD}(answer, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                      $pp_1 \leftarrow \text{next}(pp_1)$ 
19                  $p_1 \leftarrow \text{next}(p_1)$ 
20                  $p_2 \leftarrow \text{next}(p_2)$ 
21             else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                 then  $p_1 \leftarrow \text{next}(p_1)$ 
23             else  $p_2 \leftarrow \text{next}(p_2)$ 
24 return  $answer$ 
```



► **Figure 2.12** An algorithm for **proximity** intersection of postings lists p_1 and p_2 . The algorithm finds places where the two terms appear within k words of each other and returns a list of triples giving docID and the term position in p_1 and p_2 .

Positional index size

- You can compress position values/offsets: we'll talk about that later today.
- A positional index expands postings storage *substantially*.
- Nevertheless, a positional index is now standardly used because of the power and usefulness of phrase and proximity queries ... whether used explicitly or implicitly in a ranking retrieval system.

Positional index size

- Need an entry for each occurrence, not just once per document
- Index size depends on average document size 
 - Average web page has <1000 terms
 - SEC filings, books, even some epic poems ... easily 100,000 terms
- Consider a term with frequency 0.1%

Document size	Postings	Positional postings
1 000	1	1
100,000	1	100

Rules of thumb

- A positional index is 2–4 as large as a non-positional index
 - Positional index size 35–50% of volume of original text
 - Caveat: all of this holds for “English-like” languages
-
- Without a positional index, proximity, ordered, quorum and field (labels) searches will be VERY SLOW.



DEALING WITH Spelling ERRORS



Document correction

- Needed for OCR*'ed documents, transliterated names and place, spelling variations, product names, chemical names, ...
 - Correction algorithms are tuned for this: rn/m
 - Can use domain-specific knowledge
 - E.g., OCR can confuse O and D more often than it would confuse O and I (adjacent on the QWERTY keyboard, so more likely interchanged in typing).
- But also: web pages and even printed material have typos
- Goal: the dictionary contains fewer misspellings
- But often we don't change the documents and instead fix the query-document mapping

*) Optical Character Recognition (OCR) is the process of converting text on bitmaps (TIF, PNG, JPG) to ASCII/UNICODE character codes. So, basically image to text:

https://en.wikipedia.org/wiki/Optical_character_recognition

Query mis-spellings

- Our principal focus here
 - E.g., the query ***Ornellaia AND Sassiscaia***
- We can either
 - Retrieve documents indexed by the correct spelling,
OR
 - Return several suggested alternative queries with the correct spelling
 - *Did you mean ... ?* - Returns only high frequent queries. Not rare exceptions, so you do not find everything out there!

Isolated word correction

- Fundamental premise – there is a lexicon from which the correct spellings come
- Two basic choices for this
 - A standard lexicon such as
 - Webster's English Dictionary
 - An “industry-specific” lexicon – hand-maintained
 - The lexicon of the indexed corpus
 - E.g., all words on the web
 - All names, acronyms etc.
 - (Including the mis-spellings)

But what if ...

- Someone deliberately creates a non-common spelling error.
- Uses a name or place that is unique and has never seen by humanity before. You can make 20^{72} unique combinations of 8 positions, upper case, lower case, numbers and reading signs. Sure some of these will be that unique.

33WQMasUwbL@!p\$Yc@Yf

Isolated word correction

- Given a lexicon and a character sequence Q , return the words in the lexicon closest to Q
- What's "closest"?
- We'll study several alternatives
 - Edit distance (Levenshtein distance)
 - Weighted edit distance
 - n -gram overlap with Jaccard

Edit distance

- Given two strings S_1 and S_2 , the minimum number of operations to convert one to the other
- Operations are typically character-level
 - Insert, Delete, Replace, (Transposition)
- E.g., the edit distance from **dof** to **dog** is 1
 - From **cat** to **act** is 2 (Just 1 with transpose.)
 - from **cat** to **dog** is 3.
- Generally found by dynamic programming.

See: https://en.wikipedia.org/wiki/Edit_distance for various examples.

Weighted edit distance

- As above, but the weight of an operation depends on the character(s) involved
 - Meant to capture OCR or keyboard errors
Example: **m** more likely to be mis-typed as **n** than as **q**
 - Therefore, replacing **m** by **n** is a smaller edit distance than by **q**
 - This may be formulated as a probability model
- Requires weight matrix as input
- Modify dynamic programming to handle weights
- We will look at this in more detail later in the lecture on “normalization of named entities”

Using edit distances

- Given query, first enumerate all character sequences within a preset (weighted) edit distance (e.g., 2)
- Intersect this set with list of “correct” words
- Show terms you found to user as suggestions
- Alternatively,
 - We can look up all possible corrections in our inverted index and return all docs ... slow
 - We can run with a single most likely correction
- The alternatives disempower the user, but save a round of interaction with the user



Edit distance to all dictionary terms?

- Given a (mis-spelled) query – do we compute its edit distance to every dictionary term?
 - Expensive and slow
 - Alternative?
- How do we cut the set of candidate dictionary terms?
- One possibility is to use n -gram overlap for this
- This can also be used by itself for spelling correction.

n-gram overlap

- Enumerate all the *n*-grams in the query string as well as in the lexicon
- Use the *n*-gram index (see also wild-card search further on) to retrieve all lexicon terms matching any of the query *n*-grams
- Threshold by number of matching *n*-grams
 - Variants – weight by keyboard layout, etc.

Example with trigrams

- Suppose the text is ***november***
 - Trigrams are *nov, ove, vem, emb, mbe, ber.*
 - The query is ***december***
 - Trigrams are *dec, ece, cem, emb, mbe, ber.*
 - So, 3 trigrams overlap (of 6 in each term)
-
- How can we turn this into a normalized measure of overlap?

One option – Jaccard coefficient

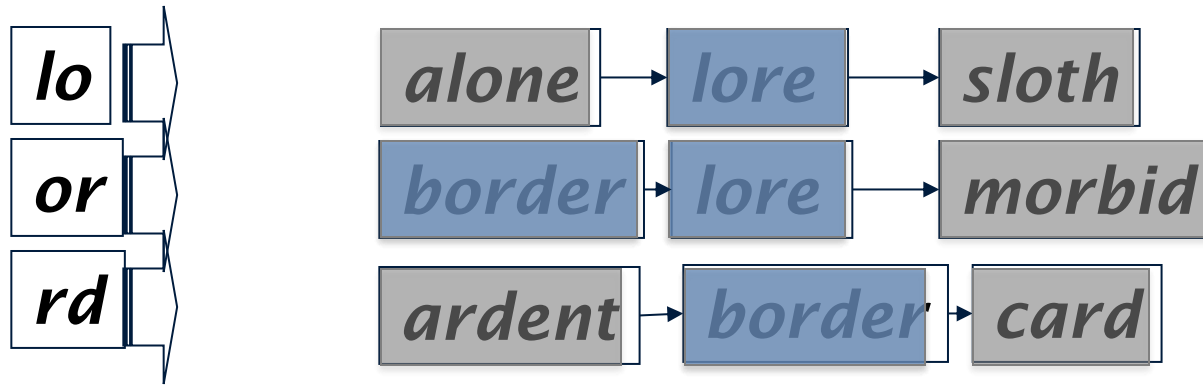
- A commonly-used measure of overlap
- Let X and Y be two sets; then the J.C. is

$$|X \cap Y| / |X \cup Y|$$

- Equals 1 when X and Y have the same elements and zero when they are disjoint
- X and Y don't have to be of the same size
- Always assigns a number between 0 and 1
 - Now threshold to decide if you have a match
 - E.g., if J.C. > 0.8 , declare a match

Matching trigrams

- Consider the query *lord* – we wish to identify words matching 2 of its 3 bigrams (*lo*, *or*, *rd*) indicated in blue.



Standard postings “merge” will enumerate ...

Adapt this to using Jaccard (or another) measure to set thresholds and avoid false positives.

There are many other alternatives

- Soundex
- Winkler
- Jaro-Winkler
- Letter-pair Similarity (Sørensen-Dice coefficient)
- Q-gram Distance
- Smith-Waterman Distance
- ...

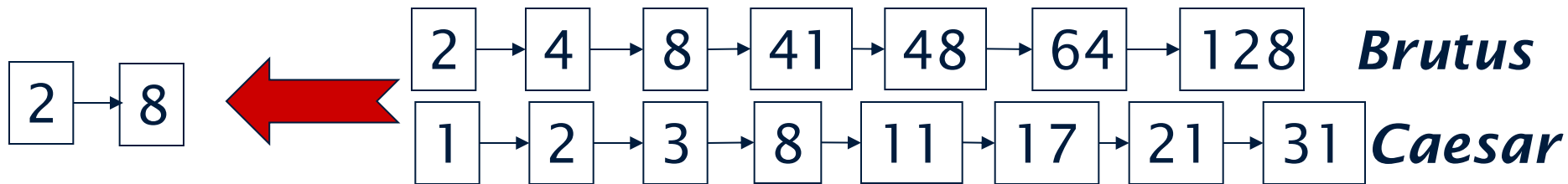
We will discuss these later as part of the “normalization” of extracted entities.

Faster postings merges: Skip pointers/Skip lists



Recall basic merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries

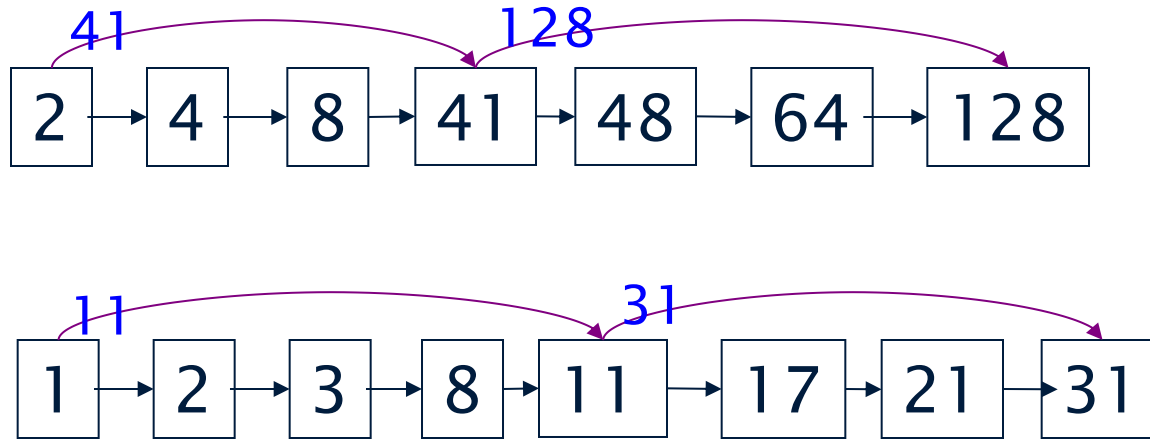


If the list lengths are m and n , the merge takes $O(m+n)$ operations.

Can we do better?

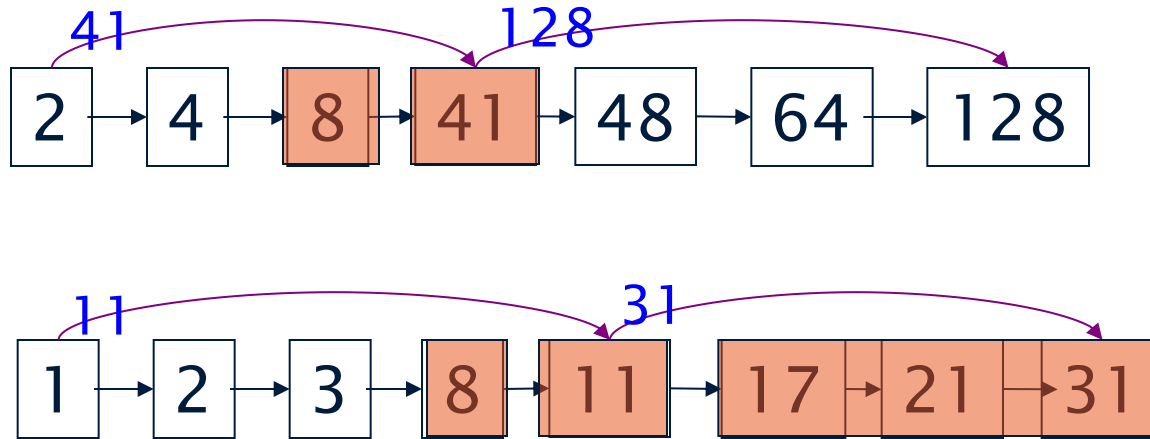
Yes (if index isn't changing too fast).

Augment postings with skip pointers (at indexing time)



- Why?
- To skip postings that will not figure in the search results.
- How?
- Where do we place skip pointers?

Query processing with skip pointers



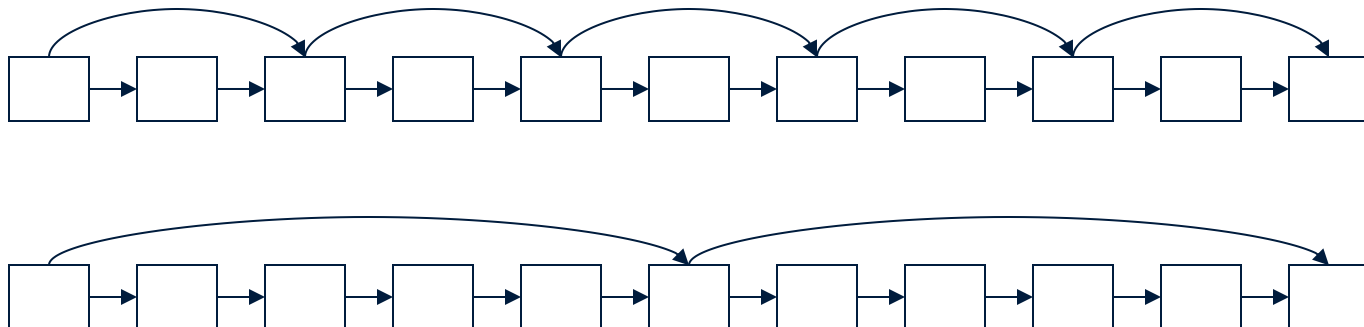
Suppose we've stepped through the lists until we process **8** on each list. We match it and advance.

We then have **41** and **11** on the lower. **11** is smaller.

But the skip successor of **11** on the lower list is **31**, so we can skip ahead past the intervening postings.

Where do we place skips?

- Tradeoff:
 - More skips $\rightarrow$ shorter skip spans $\Rightarrow$ more likely to skip. But lots of comparisons to skip pointers.
 - Fewer skips $\rightarrow$ few pointer comparison, but then long skip spans $\Rightarrow$ few successful skips.



Placing skips

- Simple heuristic: for postings of length L , use $\sqrt{L}$ evenly-spaced skip pointers.
- This ignores the distribution of query terms.
- Easy if the index is relatively static; harder if L keeps changing because of updates.

OR queries

- Why are skip pointers not useful for queries of the form $x \text{ OR } y$?

Important Characteristics of Natural Language: Heap & Zipf



RCV1: Our collection for this lecture

- The collection we'll use isn't really large enough either, but it's publicly available and is at least a more plausible example.
- As an example, for applying scalable index construction algorithms, we will use the Reuters RCV1 collection.
- This is one year of Reuters newswire (part of 1995 and 1996)
- Download: <https://trec.nist.gov/data/reuters/reuters.html>

A Reuters RCV1 document



You are here: [Home](#) > [News](#) > [Science](#) > [Article](#)

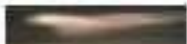
Go to a Section: [U.S.](#) [International](#) [Business](#) [Markets](#) [Politics](#) [Entertainment](#) [Technology](#) [Sports](#) [Oddly Enough](#)

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET

[Email This Article](#) | [Print This Article](#) | [Reprints](#)

[\[-\]](#) Text [\[+\]](#)



SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian meteorological base at Mawson Station on July 25.

Reuters RCV1 statistics

symbolstatistic		value
N	documents	800,000
L	avg. # tokens per doc	200
M	terms (= unique tokens)	400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	non-positional postings	100,000,000 (count, cannot calculate upfront exactly)

4.5 bytes per word token vs. 7.5

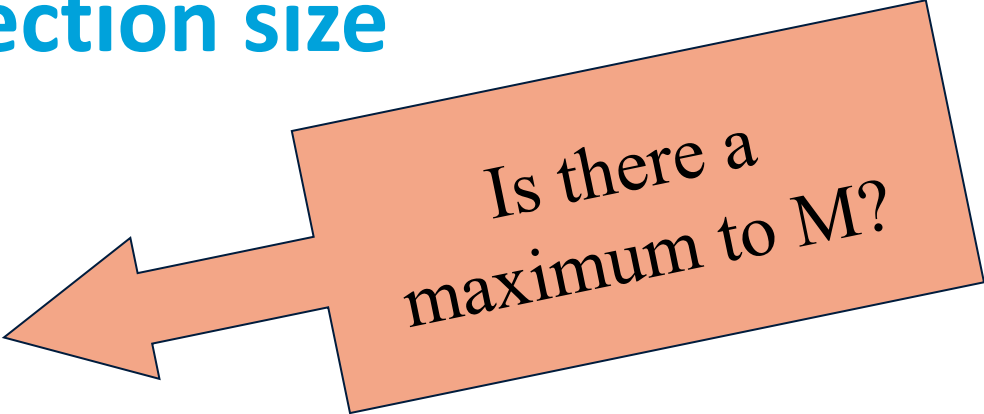
bytes per word type: why?

How many POSITIONAL tokens?

Vocabulary vs. collection size

- How big is the term vocabulary?
 - That is, how many distinct words are there?
- Can we assume an upper bound?
 - Not really: At least $70^{20} = 10^{37}$ different words of length 20
- In practice, the vocabulary will keep growing with the collection size
 - Especially with Unicode 😊

Vocabulary vs. collection size



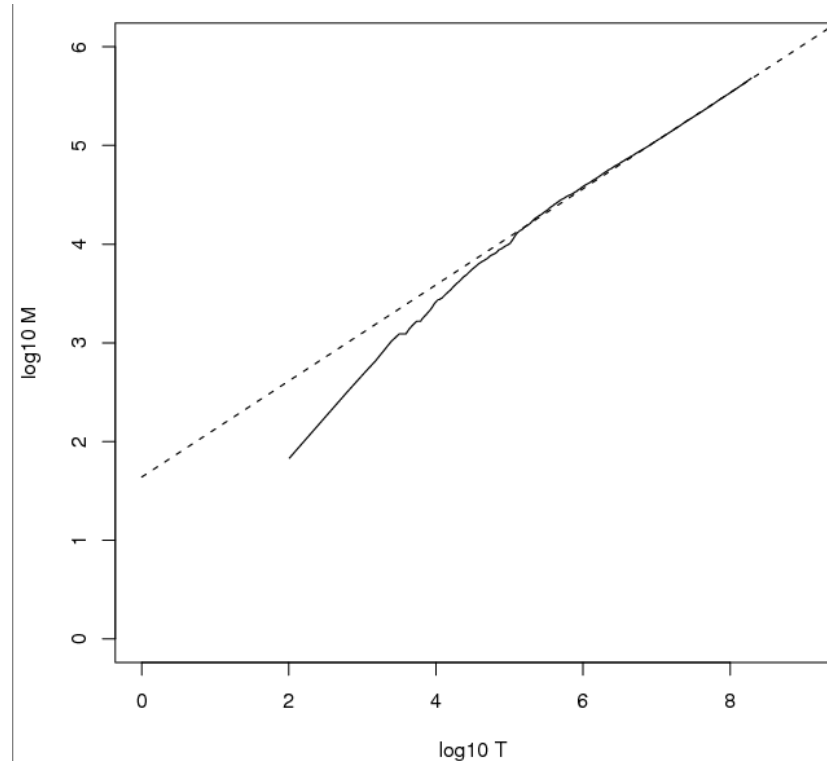
Is there a maximum to M ?

- Heaps' law: $M = kT^b$
- M is the size of the vocabulary, T is the number of tokens in the collection
- Typical values: $30 \leq k \leq 100$ and $b \approx 0.5$
- In a log-log plot of vocabulary size M vs. T , Heaps' law predicts a line with slope about $\frac{1}{2}$
 - It is the simplest possible relationship between the two in log-log space
 - An empirical finding ("empirical law")

Heaps' Law

- For RCV1, the dashed line
- $\log_{10} M = 0.49 \log_{10} T + 1.64$ is the best least squares fit.
- Thus, $M = 10^{1.64} T^{0.49}$ so $k = 10^{1.64} \approx 44$ and $b = 0.49$.
- Good empirical fit for Reuters RCV1 !
- For first 1,000,020 tokens,
- law predicts 38,323 terms;
- actually, 38,365 terms

Fig 5.1 p81



How would k and b change for collections with (i) multiple languages and (ii) OCR errors?

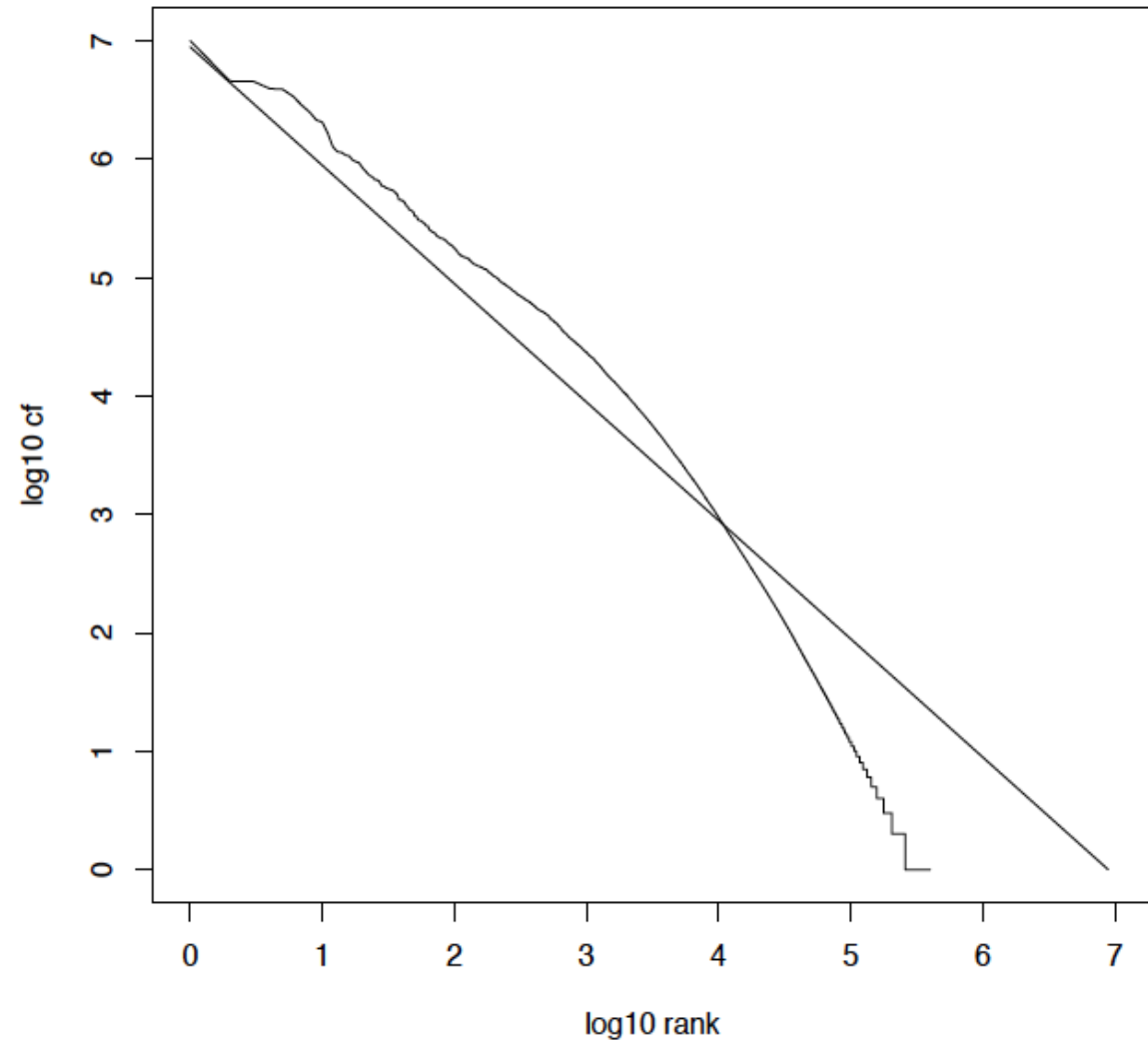
Zipf's law

- Heaps' law gives the vocabulary size in collections.
- We also study the relative frequencies of terms.
- In natural language, there are a few very frequent terms and very many very rare terms.
- Zipf's law: The i^{th} most frequent term has frequency proportional to $1/i$.
- $cf_i \propto 1/i = K/i$ where K is a normalizing constant
- cf_i is collection frequency: the number of occurrences of the term t_i in the collection.

Zipf consequences

- If the most frequent term (*the*) occurs cf_1 times
 - then the second most frequent term (*of*) occurs $cf_1/2$ times
 - the third most frequent term (*and*) occurs $cf_1/3$ times ...
- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
 - $\log cf_i = \log K - \log i$
 - Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship

Zipf's law for Reuters RCV1

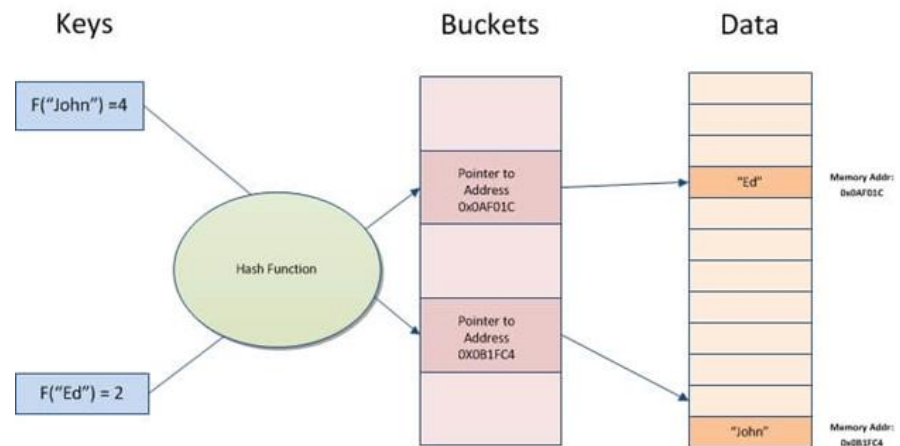
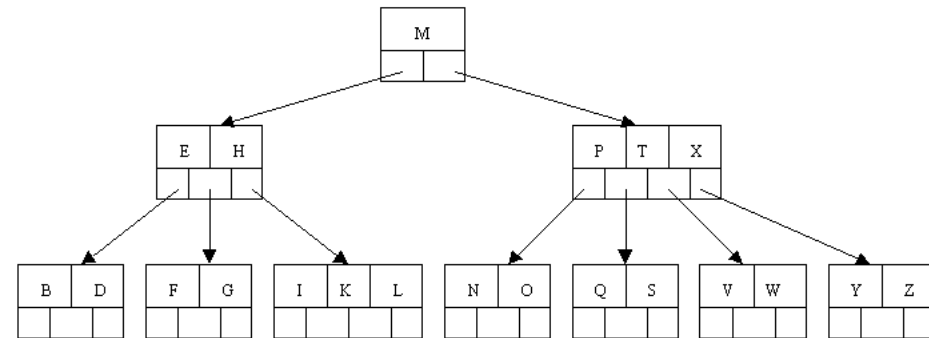


So why is this important?

- Use characteristics of language distributions to optimize storage and code:
 - Compression and fast processing
 - (encryption and deciphering)
 - Running on devices with little memory or processing capacity
 - Many NLP, clustering and machine learning algorithms are NP-Complete, so we can use all the optimizations we can come up with.
 - Anomaly detection

The core of the index

- A lookup table of the vocabulary
 - Used to be B-Trees (indexing $O(n \log n)$ with n number of unique tokens due to sorting)
 - These days: hash tables (indexing of $O(n)$, direct memory storage)
- From there on: linked lists of postings
- In the postings, also positions
- Additional tables to speed up incomplete searching (we will get to that later) or advanced search like entity search



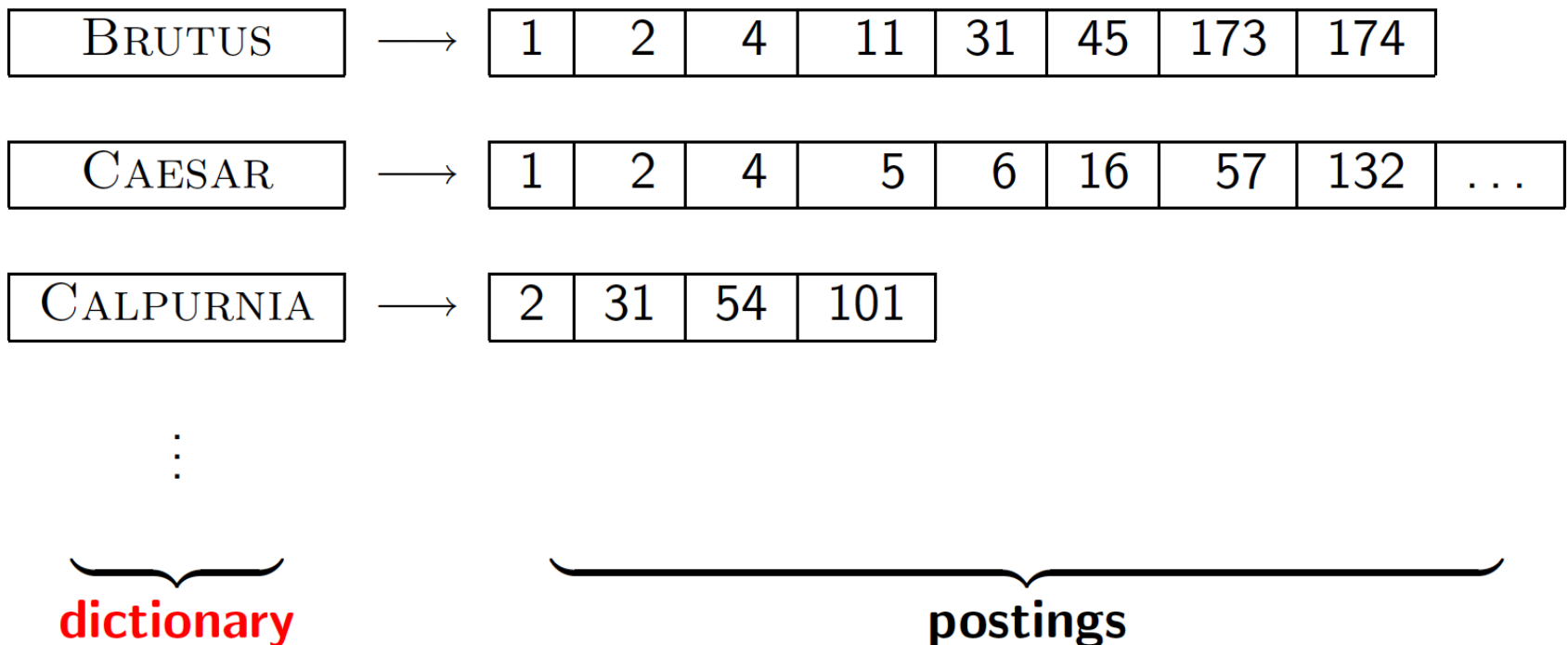
The Index Data Structures



Maastricht University

Dictionary data structures for inverted indexes

- The dictionary data structure stores the term vocabulary, document frequency, pointers to each postings list ... **in what data structure?**



A Naïve Dictionary

- An array of struct:

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

char[20] int Postings *
 20 bytes 4/8 bytes 4/8 bytes

- How do we store a dictionary in memory efficiently?
- How do we quickly look up elements at query time?

Dictionary data structures

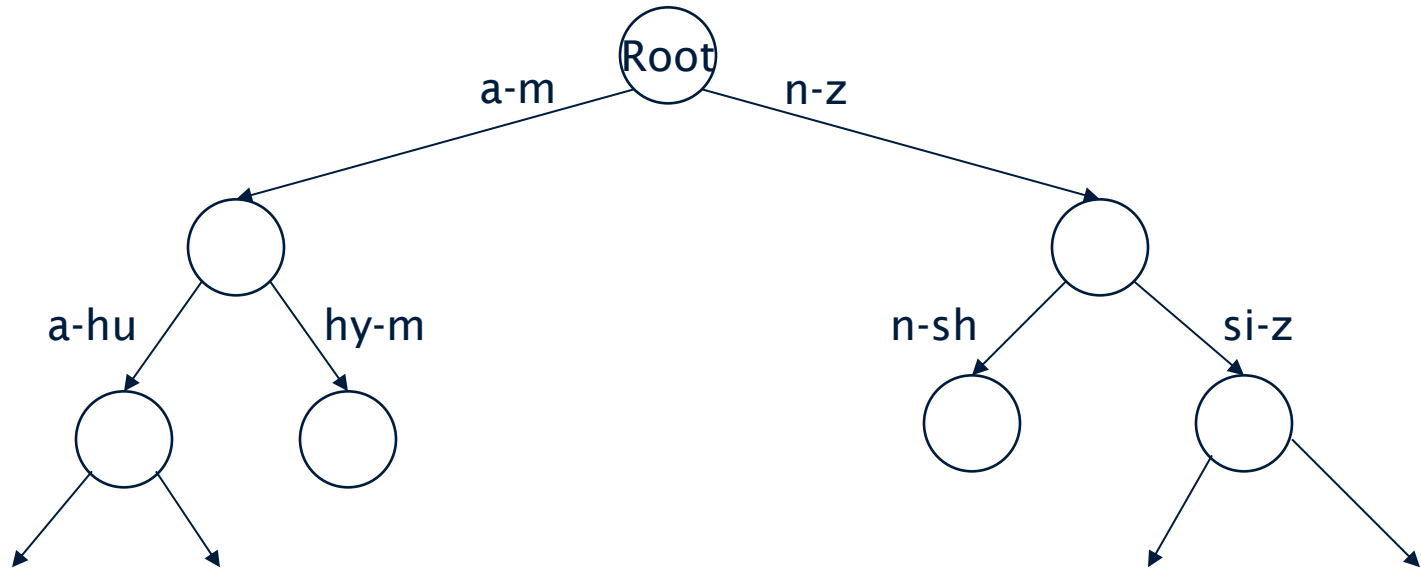
- Two main choices:
 - Hash tables
 - Trees
- Some IR systems used hash-tables, some trees. These days, always hash-tables with additional lookup tables for incomplete or semantic searches.

Hash-tables

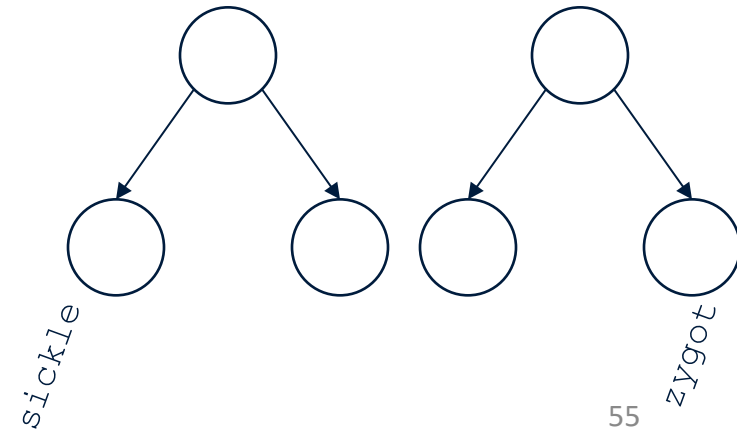
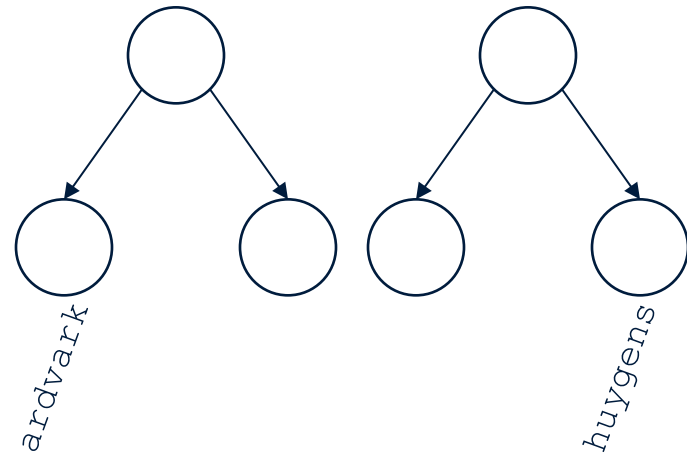
- Each vocabulary term is hashed to an integer
 - (We assume you've seen hash-tables before)
- Pros:
 - Lookup is faster than for a tree: $O(1)$
- Cons:
 - No easy way to find minor variants:
 - judgment/judgement
 - No prefix search [tolerant retrieval]
 - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing *everything*



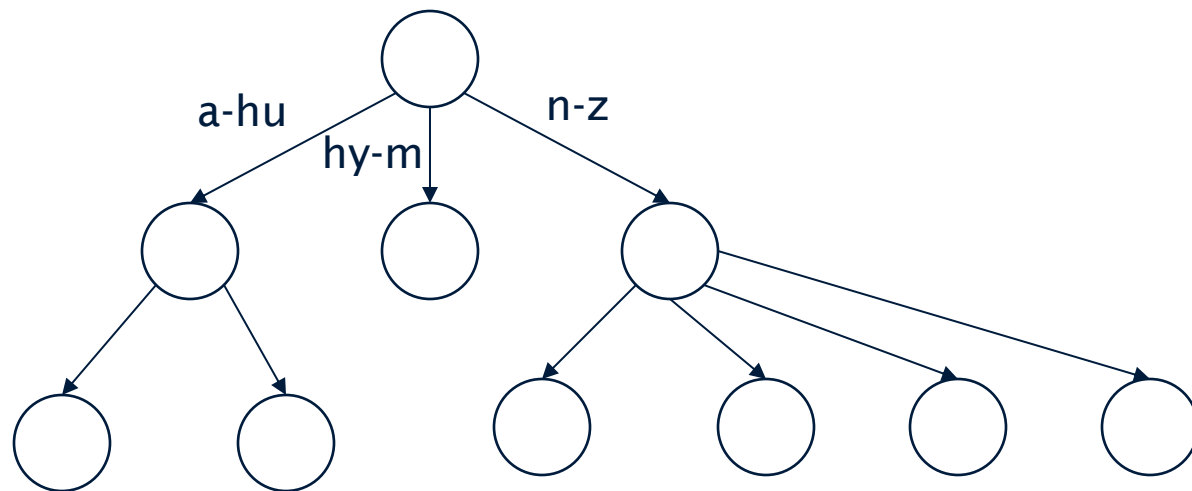
Tree: binary tree



...



Tree: B-tree



- Definition: Every internal node has a number of children in the interval $[a, b]$ where a, b are appropriate natural numbers, e.g., $[2, 4]$.

Trees

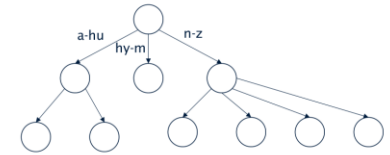
- Simplest: binary tree
- More usual: B-trees
- Trees require a standard ordering of characters and hence strings ... but we typically have one
- Pros:
 - Solves the prefix problem (terms starting with *hyp*), great for wildcard and edit distance searches
- Cons:
 - Slower: $O(N \log N)$, with N = size vocabulary [and this requires *balanced* tree]. Non-balanced tree can be $O(N \log N)$ At certain moment index merges take too long.
 - Rebalancing binary trees is expensive
 - But B-trees mitigate the rebalancing problem



Wild-card queries



Wild-card queries: *



- Definition: Every internal node has a number of children in the interval $[a, b]$ where a, b are appropriate natural numbers, e.g., $[2, 4]$.

54

- ***mon****: find all docs containing any word beginning with “mon”.
- Easy with binary tree (or B-tree) lexicon: retrieve all words in range: ***mon ≤ w < moo***
- ****mon***: find words ending in “mon”: harder
 - Maintain an additional B-tree for terms *backwards*.
Can retrieve all words in range: ***nom ≤ w < non***.

Exercise: from this, how can we enumerate all terms meeting the wild-card query ***pro*cent*** ?

Query processing

- At this point, we have an enumeration of all terms in the dictionary that match the wild-card query.
- We still have to look up the postings for each enumerated term.
- E.g., consider the query:
se*ate AND fil*er

This may result in the execution of many Boolean *AND* queries.

B-trees handle *'s at the end of a query term

- How can we handle *'s in the middle of query term?
 - *co*tion*
- We could look up *co** AND **tion* in a B-tree and intersect the two term sets
 - Expensive
- The solution: transform wild-card queries so that the *'s occur at the end
- This gives rise to the **Permuterm** Index.

Permuterm index (only suited for B-trees)

- For term *hello*, index under:
 - hello\$, ello\$h, llo\$he, lo\$hel, o\$hell*where \$ is a special symbol.

hell* = hell*\$,
rotate so * at
end, \$hell* in
b-tree, find
hello,
hellbound,

- Queries:
 - X** lookup on **X\$** **X*** lookup on **\$X***
 - *X** lookup on **X\$*** ***X*** lookup on **X***
 - X*Y** lookup on **Y\$X*** **X*Y*Z = ??????**



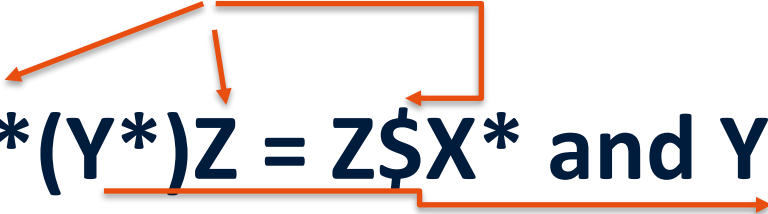
Query = *hel*o*
X=hel, Y=o
Lookup *o\$hel**

*llo = *llo\$,
rotate to llo\$*,
lookup all
elements with
rotation llo\$

The key is to rotate a wildcard query so that the * symbol appears at the end of the string. Then you can easily lookup all words with similar rotations in the B-tree.

Answer

$X^*Y = X^*Y\$$ rotate to $\$X^*Y$ and one more time to the lookup $Y\$X^*$, so 2-step:



$X^*Y^*Z = X^*(Y^*)Z = Z\$X^*$ and Y^*

So, $X^*Y^*Z = (Z\$X^*)$ and (Y^*)

Permuterm query processing

- Rotate query wild-card to the right
- Now use B-tree lookup as before.
- *Permuterm problem: $\approx$ quadruples lexicon size*



Empirical observation for English.

The diagram shows a light orange trapezoidal shape with a black outline. Inside the shape, there are two small white rectangular boxes, one on the left and one on the right, positioned near the top edge. Below the trapezoid is a solid orange rectangular box with a black border, containing the text 'Empirical observation for English.' in black font.

Another X*Y*Z Example: fi*mo*er

- First enumerate the terms in the dictionary that are in the permuterm index of **er\$fi***.
- Not all such dictionary terms will have the string **mo** in the middle - we filter these out by exhaustive enumeration, checking each candidate to see if it contains **mo**.
- In this example, the term *fishmonger* would survive this filtering but *filibuster* would not.
- We then run the surviving terms through the standard inverted index for document retrieval.

Limitations to this approach

- It only works on B-trees
- B-trees become too slow with large data sets ($n \log(n)$ complexity due to sorting at indexing).
- With a *Permuterm* index, the dictionary becomes quite large due to all rotations.
- So, the trend is towards hashing indexes with bigrams for wildcard and incomplete search.

Bigram (k -gram) indexes (best for hash-table based vocabulary indices)

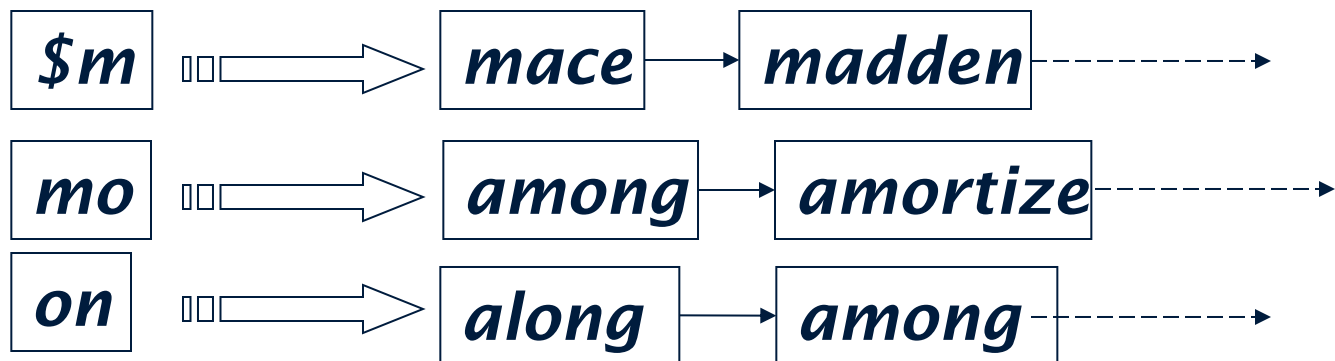
- Enumerate all k -grams (sequence of k chars) occurring in any term
- *e.g.*, from text “***April is the cruelest month***” we get the 2-grams (*bigrams*)

\$a,ap,pr,ri,il,l\$, \$i,is,s\$, \$t,th,he,e\$, \$c,cr,ru,
ue,el,le,es,st,t\$, \$m,mo,on,nt,h\$

- \$ is a special word boundary symbol
- Maintain a second inverted index from bigrams to dictionary terms that match each bigram.

Bigram index example

- The k -gram index finds *terms* based on a query consisting of k -grams (here $k=2$).



Processing wild-cards

- Query ***mon**** can now be run as
 - ***\$m AND mo AND on***
- Gets terms that match AND version of our wildcard query.
- But we'd enumerate ***moon***.
- Must post-filter these terms against query.
- Surviving enumerated terms are then looked up in the term-document inverted index.
- Fast, space efficient (compared to permuterm).

Processing wild-card queries

- As before, we must execute a Boolean query for each enumerated, filtered term.
- Wild-cards can result in expensive query execution (very large disjunctions...)
 - `pyth*` AND `prog*`
- If you encourage “laziness” people will respond!

Search

Type your search terms, use '*' if you need to.
E.g., `Alex*` will match Alexander.

- Which web search engines allow wildcard queries?

Break



Index Construction



Maastricht University

Index construction

- How do we construct an index?
- What strategies can we use with limited main memory?

Hardware basics

- Many design decisions in information retrieval are based on the characteristics of hardware
- We begin by reviewing hardware basics



Hardware basics

- Access to data in memory is ***much*** faster than access to data on disk (not every disk is an SSD!).
- Disk seeks: No data is transferred from disk while the disk head is being positioned.
- Therefore: Transferring one large chunk of data from disk to memory is faster than transferring many small chunks.
- Disk I/O is block-based: Reading and writing of entire blocks (as opposed to smaller chunks).
- Compressed data is smaller, thus also faster!

Hardware basics

- Servers used in IR systems now typically have several GB of main memory, sometimes tens of GB.
- Available disk space is several (2–3) orders of magnitude larger than RAM (e.g., 1 Tb versus 4 Gb).
- Fault tolerance is very expensive: It's much cheaper to use many regular machines rather than one fault tolerant machine.
- This is why many cloud computers use cheap components, e.g. normal (slow) hard disks.

Cost per Tb (the Scaling Problem)

- In IR, you are not dealing with 512 Gb, you are dealing with PetaBytes).
- HDD are significantly cheaper than SSD
- In the long term, SSDs “wear out” after certain number of R/W cycles.
- HDD have predictable failures (noises, slowing down), SSDs just fail.
- Many of the algorithms we will discuss later are optimized for the reading of large chunks of data as HDD do.

It is more cost effective to have many slower cheap machines than less highly expensive redundant machines. The redundancy is in the software and not in the hardware.

Recall: Reuters RCV1 statistics

symbolstatistic		value
N	documents	800,000
L	avg. # tokens per doc	200
M	terms (= unique tokens)	400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	non-positional postings	100,000,000 (count, cannot calculate upfront exactly)
	positional postings	160,000,000

Recall index construction

- Documents are parsed to extract words and these are saved with the Document ID.

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious



Term	Doc #
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Key step

- After all documents have been parsed, the inverted file is sorted by terms.

We focus on this sort step.
We have 100M items to sort.

Term	Doc #	Term	Doc #
I	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
I	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	I	1
killed	1	I	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	2
noble	2	so	2
brutus	2	the	1
hath	2	the	2
told	2	told	2
you	2	you	2
caesar	2	was	1
was	2	was	2
ambitious	2	with	2

Scaling index construction

- In-memory index construction does not scale
 - Can't stuff entire collection into memory, sort, then write back
- How can we construct an index for very large collections?
- Taking into account the hardware constraints we just learned about . . .
- Memory, disk, speed, etc.

Sort-based index construction

Could be done
in parallel



- As we build the index, we parse docs one at a time.
 - While building the index, we cannot easily exploit compression tricks (you can, but much more complex)
- The final postings for any term are incomplete until the end.
- At 12 bytes per non-positional postings entry (*term, doc, freq*), demands a lot of space for large collections.
- $T = 100,000,000$ in the case of RCV1
 - So ... we can do this in memory in 2026, but typical collections are much larger. E.g., the *New York Times* provides an index of >150 years of newswire
- Thus: We need to store intermediate results on disk. 

Always count
on this!

You will always run out of RAM

Metric	Reuters RCV1	NYT (150 Years)	WSJ (Historical)
Documents	800,000	~15–20 Million	~15–18 Million
Non-Positional Postings	100 Million	~2.5 Billion	~2.2 Billion
Memory (Non-Pos)	1.2 GB	~30 GB	~26.4 GB
Positional Indexing	~6–8 GB	>500 GB	~450 GB
Processing Strategy	In-Memory	Disk-Based (BSBI)	Disk-Based (BSBI)

RCV1 is the "small" example used to teach the basics because it fits in a laptop's RAM. WSJ and NYT represent "real-world" scale where you **must** use the disk-based sorting methods shown later in this lecture.

Sort using disk as “memory”?

- Can we use the same index construction algorithm for larger collections, but by using disk instead of memory?
- No: Sorting $T = 100,000,000$ records on disk is too slow – too many disk seeks.
- We need an *external* sorting algorithm.

Bottleneck

- Parse and build postings entries one doc at a time
- Now sort postings entries by term (then by doc within each term)
- Doing this with random disk seeks would be too slow – must sort $T=100M$ records



If every comparison took 2 disk seeks, and N items could be sorted with $N \log_2 N$ comparisons, how long would this take?

Calculations

- If we need $n \log_2 n$ comparisons (where n is the number of termID-docID pairs) and 2 disk seeks for each comparison, how much time would index construction for Reuters-RCV1 take if we used disk instead of memory for storage and an unoptimized sorting algorithm (i.e., not an external sorting algorithm)?
- For Reuters $N = 100$ million = 10^8
- 2 seeks
- $N \log_2 N$
- So: $2 \times 10^8 \times \log_2 10^8 \times$ disk seek and access time = a lot!



5 to 10 milliseconds for every lookup! We need (tens of) thousands or even more lookups for large Booleans, wildcard or fuzzy queries.

BSBI: Blocked sort-based Indexing (Sorting with fewer disk seeks)

- 12-byte (4+4+4) records (*term*, *doc*, *freq*).
- These are generated as we parse docs.
- Must now sort 100M such 12-byte records by *term*.
- Define a Block ~ 10M such records
 - Can easily fit a couple into memory.
 - Will have 10 such blocks to start with.
- Basic idea of algorithm:
 - Accumulate postings for each block, sort, write to disk.
 - Then merge the blocks into one long sorted order.



Sequential



Parallel

postings
to be merged

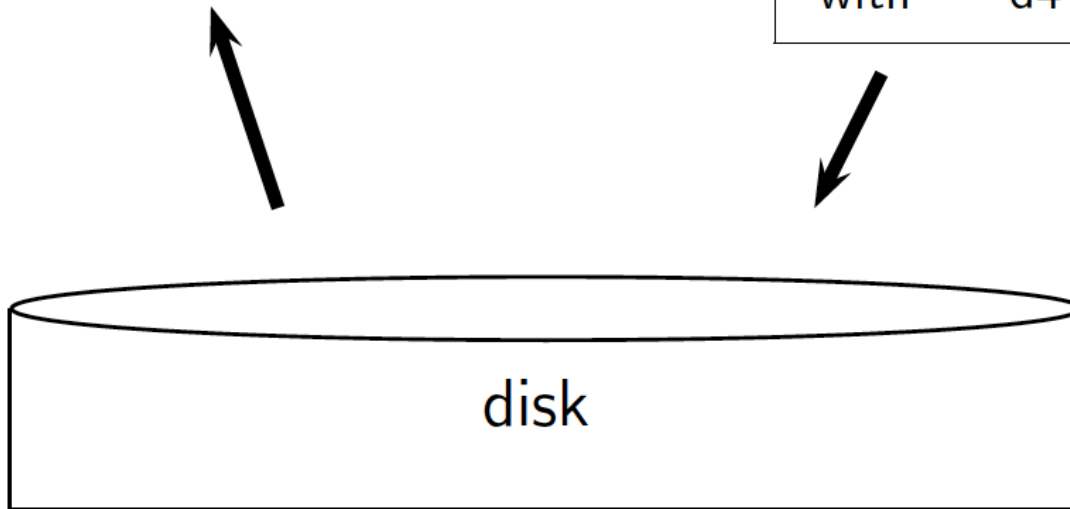
brutus	d3
caesar	d4
noble	d3
with	d4

brutus	d2
caesar	d1
julius	d1
killed	d2



brutus	d2
brutus	d3
caesar	d1
caesar	d4
julius	d1
killed	d2
noble	d3
with	d4

merged
postings



Sorting 10 blocks of 10M records

- First, read each block and sort within:
 - Quicksort takes $2N \ln N$ expected steps
 - In our case $2 \times (10M \ln 10M)$ steps
- 10 times this estimate – gives us 10 sorted runs of 10M records each.
- Done straightforwardly, need 2 copies of data on disk
 - But can optimize this.

In $n \log(n)$ complexity this saves us about 12% for 100M element, But it increases significantly for larger numbers or for $O(n^2)$ problems.

Cutting up processes that take more than linear time.

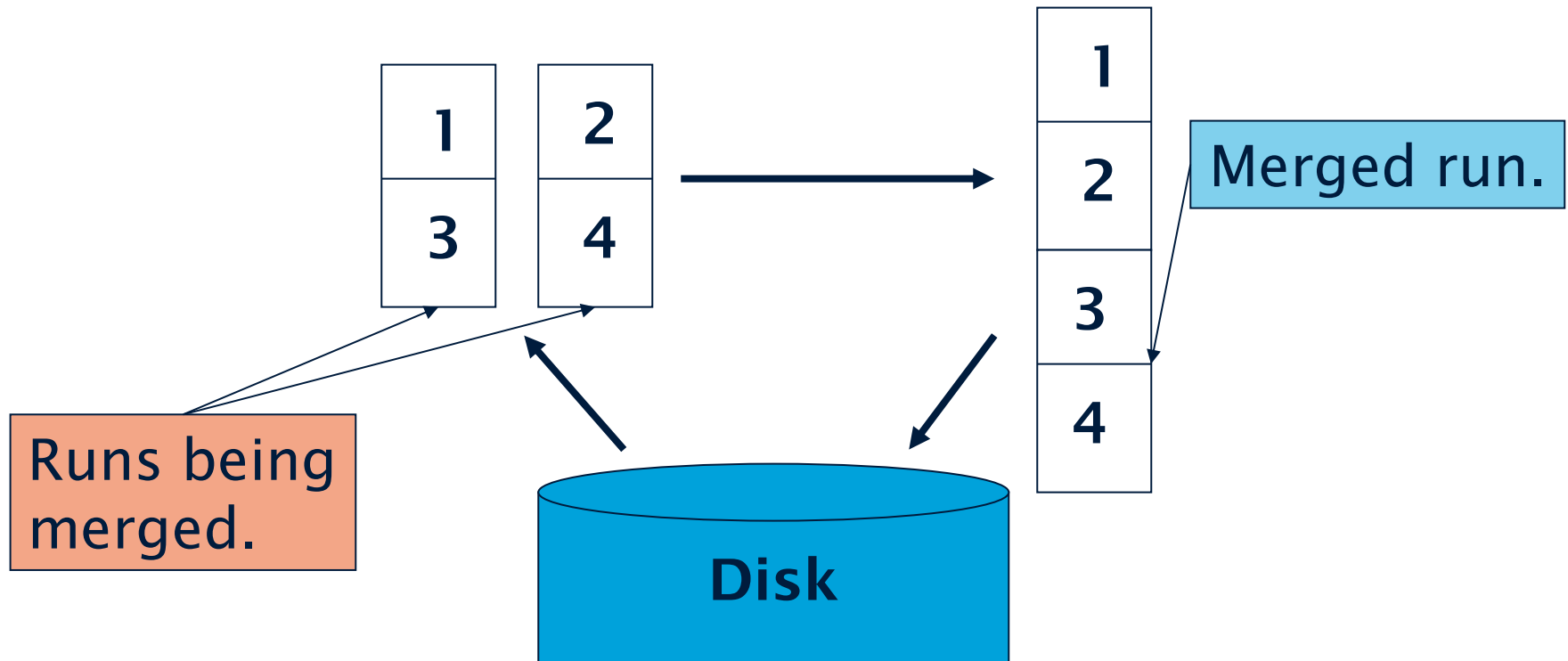
- Sorting 100,000 books by wandering across a massive library is exhausting because every new book requires you to look past thousands of others ($O(n^2)$). But if you divide the library into 10 small zones, you only have to sort 10,000 books at a time. The pile is smaller, you can see everything at once, and the "penalty" of the massive scale disappears.
- When you break a massive task into smaller chunks, non-linear algorithms benefit exponentially because you are dodging the steepest part of the math curve. To see exactly how many operations you save by splitting 100 million items into 10 chunks (or even 50 chunks!), try adjusting the parameters in the visualizer below.

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

How to merge the sorted runs?

- Can do binary merges, with a merge tree of $\log_2 10 = 4$ layers.
- During each layer, read into memory runs in blocks of 10M, merge, write back.



How to merge the sorted runs?

- But it is more efficient to do a multi-way merge, where you are reading from all blocks simultaneously
- Providing you read decent-sized chunks of each block into memory and then write out a decent-sized output chunk, then you're not killed by disk seeks

Remaining problem with sort-based algorithm

- Our assumption was: we can keep the dictionary in memory.
- We need the dictionary (which grows dynamically) in order to implement a term to termID mapping.
- Actually, we could work with term,docID postings instead of termID,docID postings . . .
... but then intermediate files become very large. (We would end up with a scalable, but very slow index construction method.)

SPIMI: Single-pass in-memory indexing

- Key idea 1: Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- Key idea 2: Don't sort. Accumulate postings in postings lists as they occur.
- With these two ideas we can generate a complete inverted index for each block.
- These separate indexes can then be merged into one big index.

SPIMI-Invert

- Merging of blocks is analogous to BSBI.

```
SPIMI-INVERT(token_stream)
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token  $\leftarrow$  next(token_stream)
5      if term(token)  $\notin$  dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8          if full(postings_list)
9              then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10         ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms  $\leftarrow$  SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

SPIMI: Compression

- Compression makes SPIMI even more efficient.
 - Compression of terms
 - Compression of postings
- Stay tuned, we will discuss later today

Distributed indexing

- For web-scale indexing (don't try this at home!):
must use a distributed computing cluster
- Individual machines are fault-prone
 - Can unpredictably slow down or fail
- How do we exploit a pool of cloud machines?

Parallel / Sequential

- Parallel: Text extraction, Parsing, Tokenization, Create intermediate files, sorting of such files.
- Mostly Sequential: Merging
- But: a sequential sorting algorithm may not be efficient enough when we have to sort a huge volume of data. Therefore, parallel algorithms for sub tasks can be used. But merging remains the main bottleneck.

Web search engine data centers

- Web search data centers (Google, Bing, Baidu) mainly contain commodity machines.
- Data centers are distributed around the world.
- Estimate: Google ~1 million servers, 3 million processors/cores (Gartner 2007)
- 2026: 15-20 million physical servers, but old metrics no longer work.

Comparing 2007 to 2026

Feature	2007 (Gartner)	2026 (Industry Estimate)
Server Count	~1 Million	~15–20 Million
Primary Compute	Commodity CPUs	Custom AI Accelerators (TPUs)
Global Footprint	Dozens of sites	130+ Zones / 43 Regions
Storage Technology	Mostly HDDs	NVMe SSDs (Hot) / HDDs (Cold)
Main Workload	Indexing & Ranking	Large Language Model (LLM) Inference

In the 2000s, we measured data centers by the number of boxes. Today, we measure them by Megawatts (MW). Scale: A single modern "AI Campus" can consume 500 MW to 1 GW of power—enough to power a medium-sized city.

Distributed indexing

- Maintain a *master* machine directing the indexing job – considered “safe”.
- Break up indexing into sets of (parallel) tasks.
- Master machine assigns each task to an idle machine from a pool.

Parallel tasks

- We will use two sets of parallel tasks
 - Parsers
 - Inverters
- Break the input document collection into *splits*
- Each split is a subset of documents (corresponding to blocks in BSBI/SPIMI)

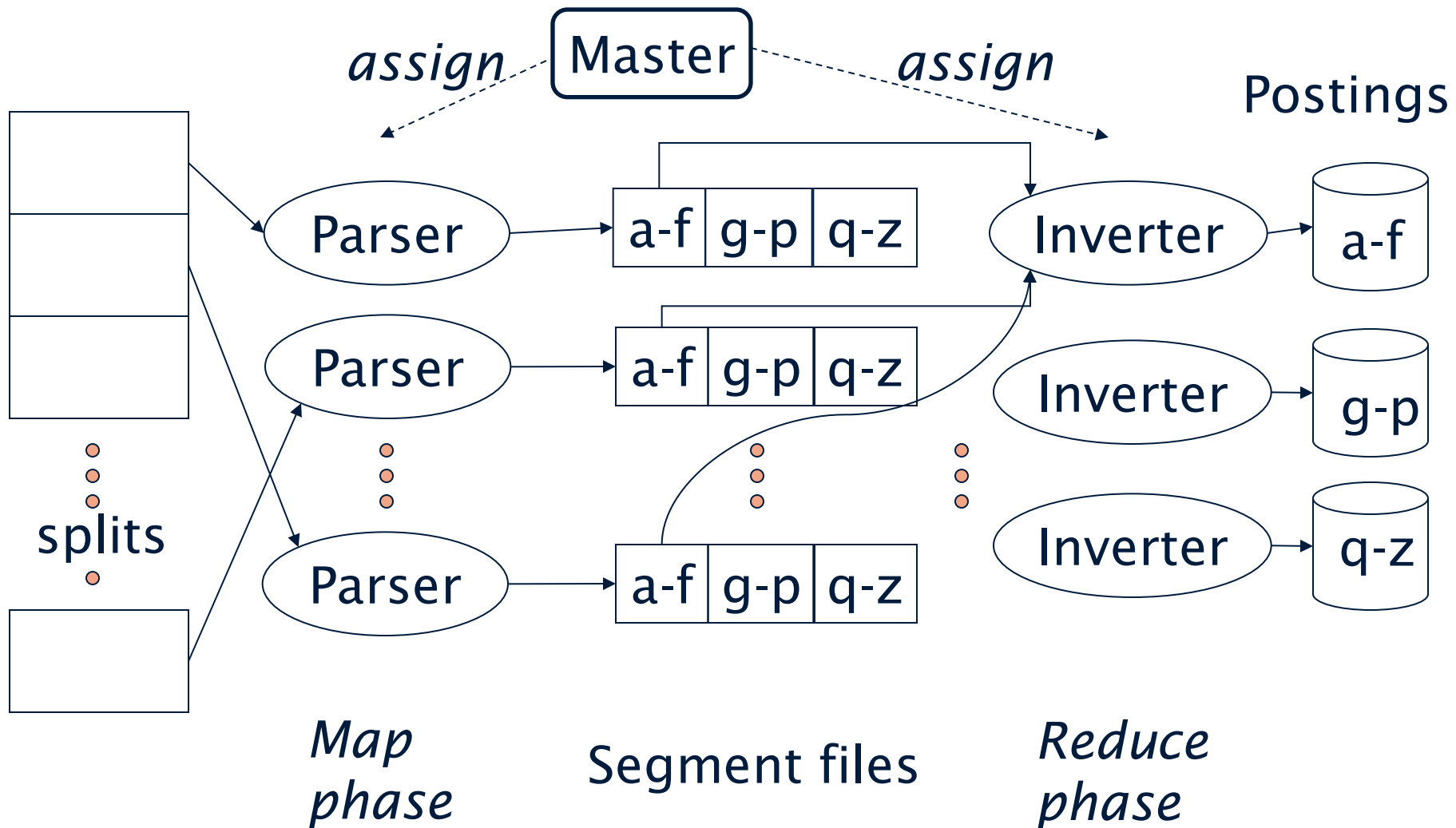
Parsers

- Master assigns a split to an idle parser machine
- Parser reads a document at a time and emits (term, doc) pairs
- Parser writes pairs into j partitions
- Each partition is for a range of terms' first letters
 - (e.g., **a-f**, **g-p**, **q-z**) – here $j = 3$.
- Now to complete the index inversion

Inverters

- An inverter collects all (term,doc) pairs (= postings) for one term-partition.
- Sorts and writes to postings lists

Data flow



MapReduce

- The index construction algorithm we just described is an instance of *MapReduce*.
- MapReduce (Dean and Ghemawat 2004) is a robust and conceptually simple framework for distributed computing ...
- ... without having to write code for the distribution part.
- They describe the Google indexing system (ca. 2002) as consisting of a number of phases, each implemented in MapReduce.

Dynamic indexing

- Up to now, we have assumed that collections are static.
- They rarely are:
 - Documents come in over time and need to be inserted.
 - Documents are deleted and modified.
- This means that the dictionary and postings lists have to be modified:
 - Postings updates for terms already in dictionary
 - New terms added to dictionary

Simplest approach

- Maintain “big” main index
- New docs go into “small” auxiliary index
- Search across both, merge results
- Deletions
 - Invalidation bit-vector for deleted docs
 - Filter docs output on a search result by this invalidation bit-vector
- Periodically, re-index into one main index

Optimizing (compressing) the index



Why compression (in general)?

- Use less disk space
 - Saves a little money
- Keep more stuff in memory
 - Increases speed
- Increase speed of data transfer from disk to memory
 - [read compressed data | decompress] is faster than [read uncompressed data]
 - Premise: Decompression algorithms are fast
 - True of the decompression algorithms we use

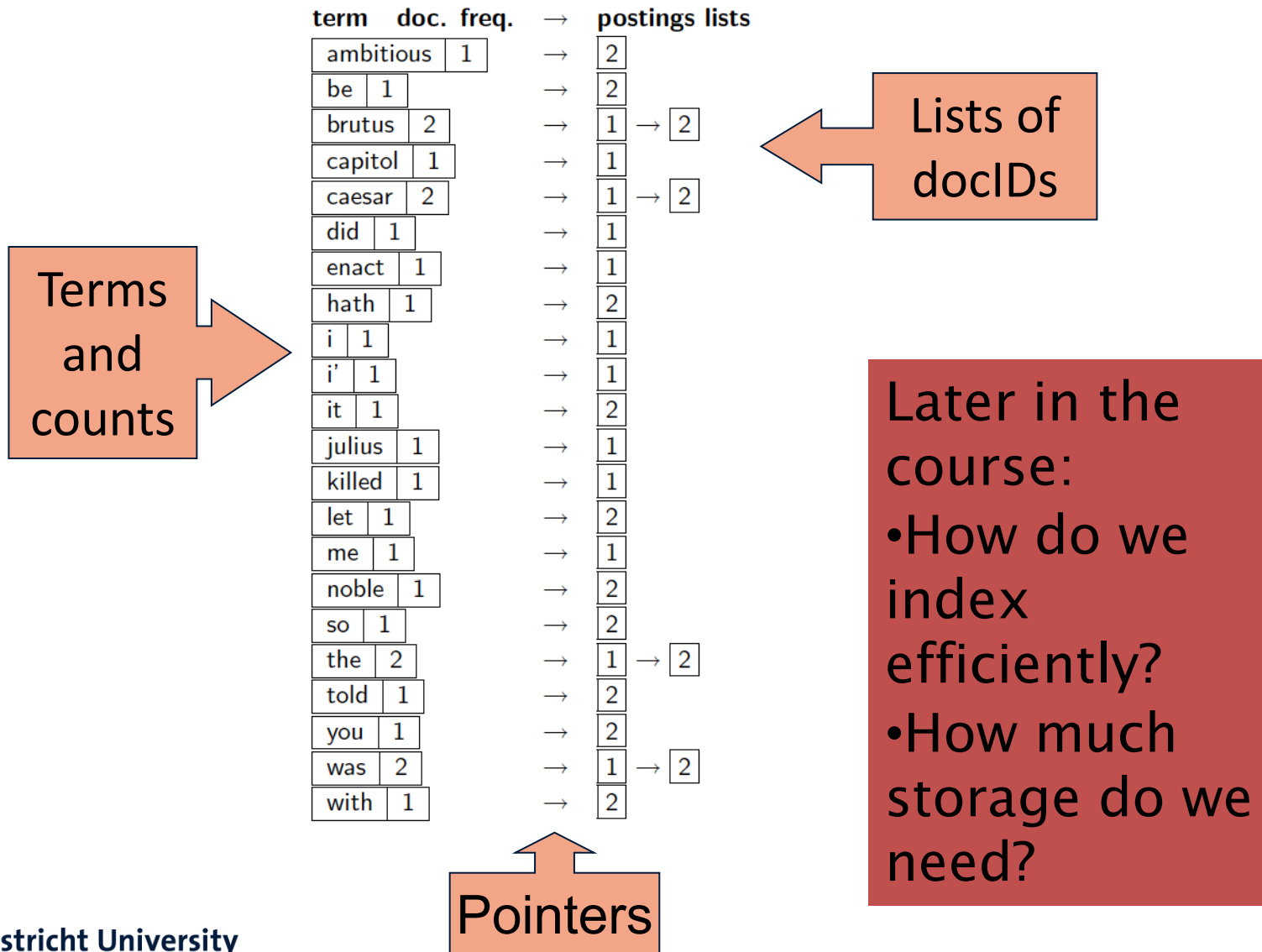
Why compression for inverted indexes?

- Dictionary
 - Make it small enough to keep in main memory
 - Make it so small that you can keep some postings lists in main memory too
- Postings file(s)
 - Reduce disk space needed
 - Decrease time needed to read postings lists from disk
 - Large search engines keep a significant part of the postings in memory.
 - Compression lets you keep more in memory
- We will devise various IR-specific compression schemes

Recall Reuters RCV1

symbol	statistic	value
N	documents	800,000
L	avg. # tokens per doc	200
M	terms (= word types)	~400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	non-positional postings	100,000,000

Where do we pay in storage?



Index parameters vs. what we index (details IIR Table 5.1, p.80)

size of	word types (terms)			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %
Unfiltered	484			109,971			197,879		
No numbers	474	-2	-2	100,680	-8	-8	179,158	-9	-9
Case folding	392	-17	-19	96,969	-3	-12	179,158	0	-9
30 stopwords	391	-0	-19	83,390	-14	-24	121,858	-31	-38
150 stopwords	391	-0	-19	67,002	-30	-39	94,517	-47	-52
stemming	322	-17	-33	63,812	-4	-42	94,517	0	-52

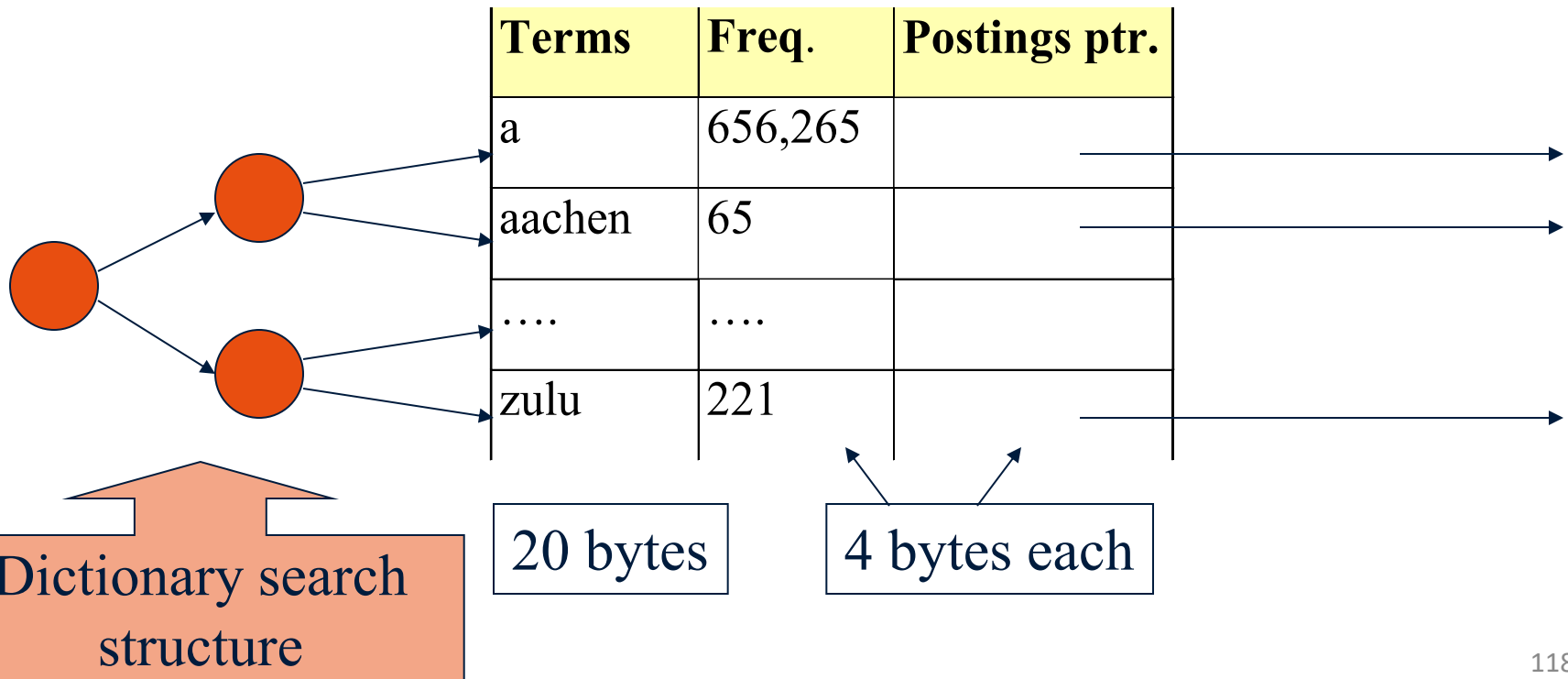
DICTIONARY COMPRESSION

Why compress the dictionary?

- Search begins with the dictionary
- We want to keep it in memory
- Memory footprint competition with other applications
- Embedded/mobile devices may have very little memory
- Even if the dictionary isn't in memory, we want it to be small for a fast search startup time
- So, compressing the dictionary is important

Dictionary storage - first cut

- Array of fixed-width entries
 - ~400,000 terms; 28 bytes/term = 11.2 MB.

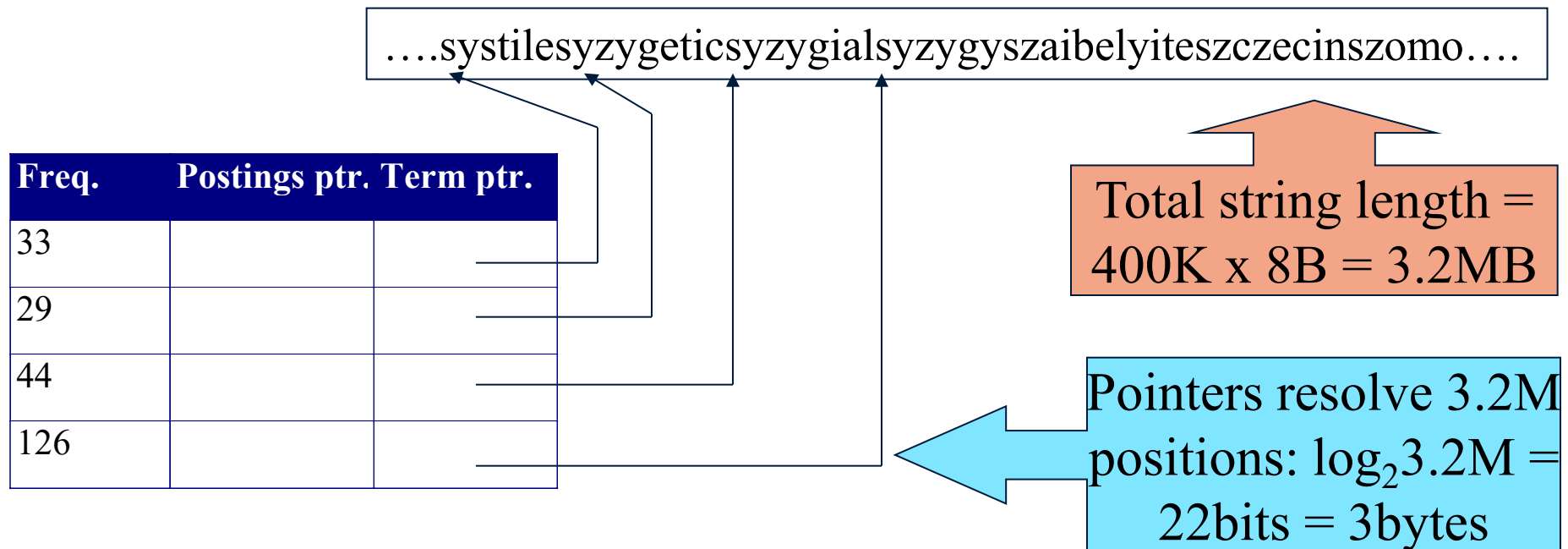


Fixed-width terms are wasteful

- Most of the bytes in the **Term** column are wasted – we allot 20 bytes for 1 letter terms.
 - And we still can't handle *supercalifragilisticexpialidocious* or *hydrochlorofluorocarbons*.
- Written English averages ~4.5 characters/word.
 - Exercise: Why is/isn't this the number to use for estimating the dictionary size?
- Ave. dictionary word in English: ~8 characters
 - How do we use ~8 characters per dictionary term?
- Short words dominate token counts but not type average.

Compressing the term list: Dictionary-as-a-String

- Store dictionary as a (long) string of characters:
 - Pointer to next word shows end of current word
 - Hope to save up to 60% of dictionary space.



Space for dictionary as a string

- 4 bytes per term for Freq.
- 4 bytes per term for pointer to Postings.
- 3 bytes per term pointer
- Avg. 8 bytes per term in term string
- 400K terms x 19 $\Rightarrow$ 7.6 MB (against 11.2MB for fixed width)

} Now avg. 11
} bytes/term,
} not 20.

Blocking

- Store pointers to every k th term string.
 - Example below: $k=4$.
- Need to store term lengths (1 extra byte)

....**7***systile***9***syzygetic***8***syzygial***6***syzygy***11***szaibelyite***8***szczecin***9***szomo*....

Freq.	Postings ptr.	Term ptr.
33		
29		
44		
126		
7		

} Save 9 bytes
on 3
pointers.

← Lose 4 bytes on
term lengths.

Net

- Example for block size $k = 4$
 - Where we used 3 bytes/pointer without blocking
 - $3 \times 4 = 12$ bytes,
- now we use $3 + 4 = 7$ bytes.

Shaved another ~ 0.5 MB. This reduces the size of the dictionary from 7.6 MB to 7.1 MB.

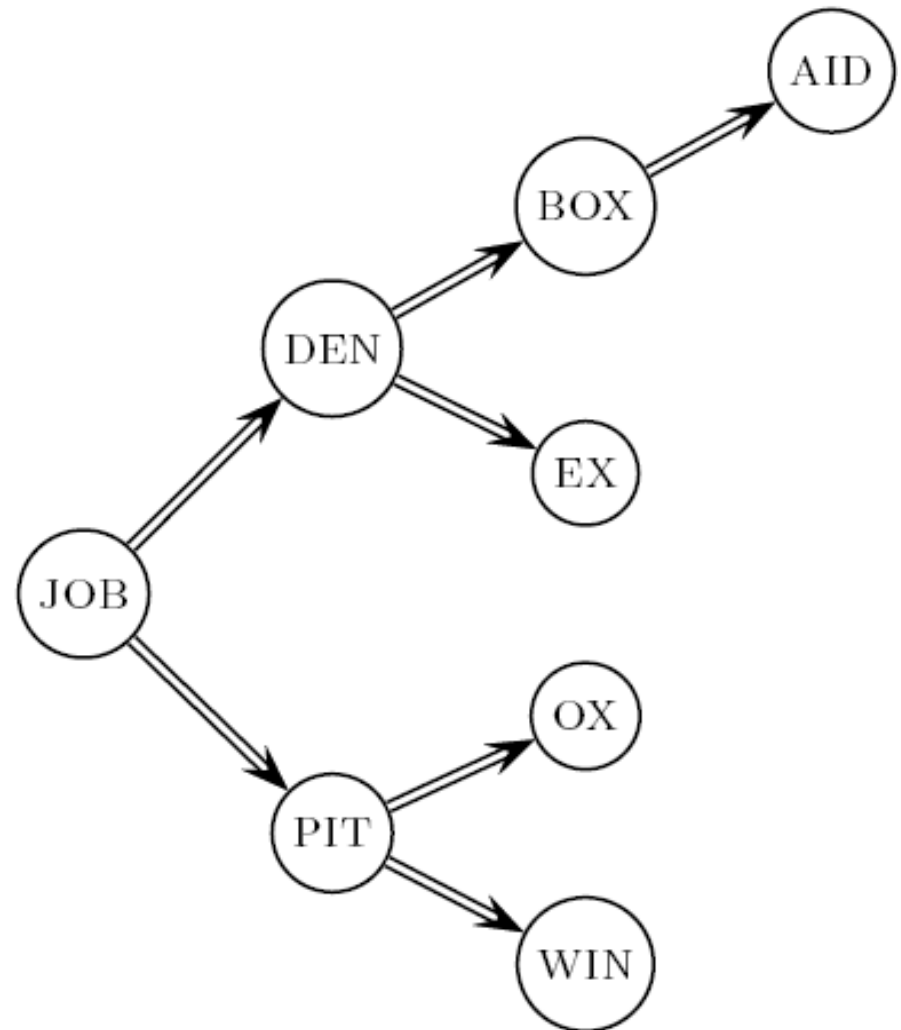
We can save more with larger k .

Why not go with larger k ?

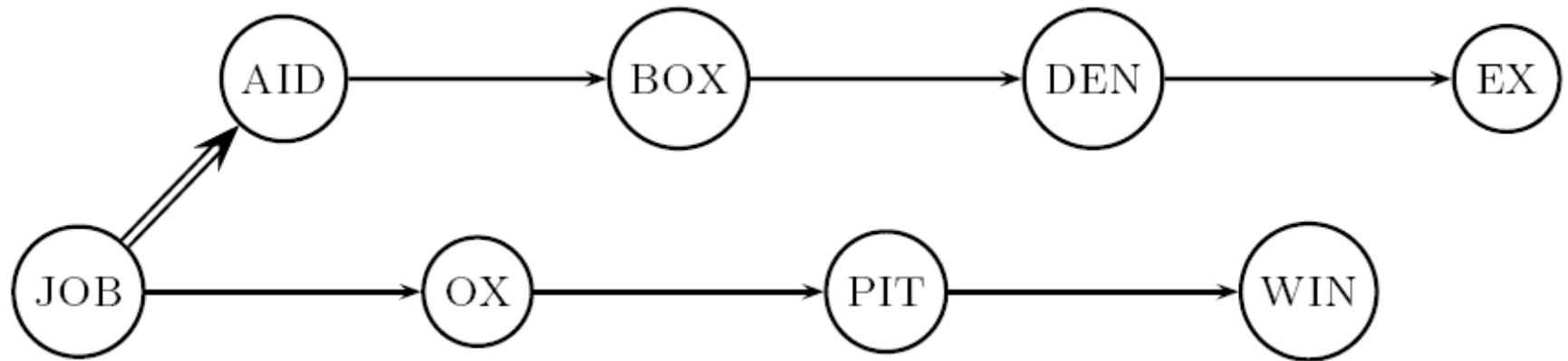
Dictionary search without blocking

- Assuming each dictionary term equally likely in query (not really so in practice!), average number of comparisons = $(1+2 \cdot 2+4 \cdot 3+4)/8 \sim 2.6$

Exercise: what if the frequencies of query terms were non-uniform but known, how would you structure the dictionary search tree?



Dictionary search with blocking



- Binary search down to 4-term block;
 - Then linear search through terms in block.
- Blocks of 4 (binary tree), avg. = $(1+2 \cdot 2+2 \cdot 3+2 \cdot 4+5)/8 = 3$ compares

Front coding

- Front-coding:
 - Sorted words commonly have long common prefix – store differences only
 - (for last $k-1$ in a block of k)

8*automata***8***automate***9***automatic***10***automation*

→ **8***automat****a****1**◇**e****2**◇**ic****3**◇**ion**

Encodes *automat*

Extra length
beyond *automat*.

Begins to resemble general string compression.¹²⁶

RCV1 dictionary compression summary

Technique	Size in MB
Fixed width	11.2
Dictionary-as-String with pointers to every term	7.6
Also, blocking $k = 4$	7.1
Also, Blocking + front coding	5.9

POSTINGS COMPRESSION

Postings compression

- The postings file is much larger than the dictionary, factor of at least 10.
- Key desideratum: store each posting compactly.
- A posting for our purposes is a docID.
- For Reuters (800,000 documents), we would use 32 bits per docID when using 4-byte integers.
- Alternatively, we can use $\log_2 800,000 \approx 20$ bits per docID.
- Our goal: use far fewer than 20 bits per docID.

Postings: two conflicting forces

- A term like ***arachnocentric*** occurs in maybe one doc out of a million – we would like to store this posting using $\log_2 1M \sim 20$ bits.
- A term like ***the*** occurs in virtually every doc, so 20 bits/posting is too expensive.
 - Prefer 0/1 bitmap vector in this case

REMEMBER ZIPF and HEAP

Other variable unit codes

- Instead of bytes, we can also use a different “unit of alignment”: 32 bits (words), 16 bits, 4 bits (nibbles).
- Variable byte alignment wastes space if you have many small gaps – nibbles do better in such cases.
- Variable byte codes:
 - Used by many commercial/research systems
 - Good low-tech blend of variable-length coding and sensitivity to computer memory alignment matches (vs. bit-level codes, which we look at next).
- There is also recent work on word-aligned codes that pack a variable number of gaps into one word

Postings file entry

- We store the list of docs containing a term in increasing order of docID.
 - **computer**: 33,47,154,159,202 ...
- Consequence: it suffices to store *gaps*.
 - 33,14,107,5,43 ...
- Hope: most gaps can be encoded/stored with far fewer than 20 bits.

Three postings entries

	encoding	postings list					
THE	docIDs	...	283042	283043	283044	283045	...
	gaps		1	1	1		...
COMPUTER	docIDs	...	283047	283154	283159	283202	...
	gaps		107	5	43		...
ARACHNOCENTRIC	docIDs	252000	500100				
	gaps	252000	248100				

Variable length encoding

- Aim:
 - For *arachnocentric*, we will use ~ 20 bits/gap entry.
 - For *the*, we will use ~ 1 bit/gap entry.
- If the average gap for a term is G , we want to use $\sim \log_2 G$ bits/gap entry.
- Key challenge: encode every integer (gap) with about as few bits as needed for that integer.
- This requires a *variable length encoding*
- Variable length codes achieve this by using short codes for small numbers

Index compression summary

- We can now create an index for highly efficient Boolean retrieval that is very space efficient
- Only 4% of the total size of the collection
- Only 10-15% of the total size of the text in the collection
- However, we've ignored positional information
- Hence, space savings are less for indexes used in practice
 - But techniques substantially the same.

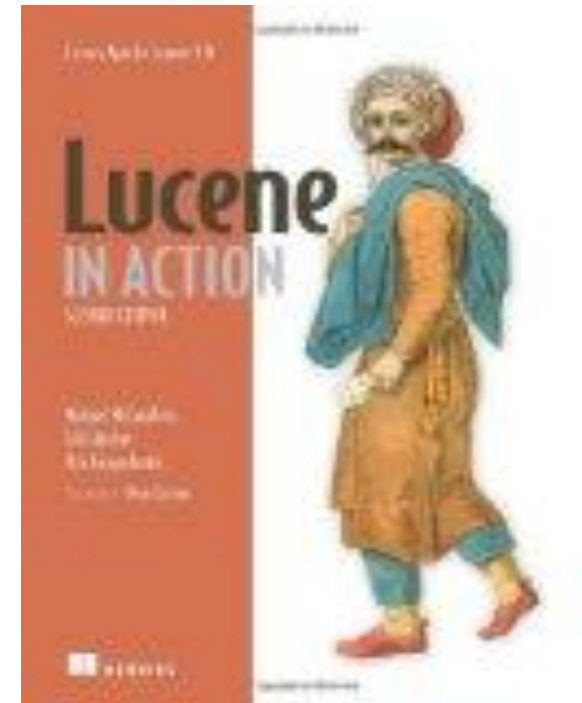
RCV1 compression

Data structure	Size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
with blocking, $k = 4$	7.1
with blocking & front coding	5.9
collection (text, xml markup etc)	3,600.0
collection (text)	960.0
Term-doc incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ -encoded	101.0

Try it yourself: Lucene

The Apache Lucene™ project develops open-source search software. The project releases a core search library, named Lucene™ core, as well as PyLucene, a python binding for Lucene.

- <https://www.lucenetutorial.com/>
- <https://lucene.apache.org/>



Resources for today's lecture

- Chapter 4 of Introduction to Information Retrieval
- Managing Gigabytes Chapter 5
- Original publication on MapReduce: Dean and Ghemawat (2004)
- Original publication on SPIMI: Heinz and Zobel (2003)

Resources for today's lecture

- *IIR 5*
- *MG 3.3, 3.4.*
- F. Scholer, H.E. Williams and J. Zobel. 2002. Compression of Inverted Indexes For Fast Query Evaluation. *Proc. ACM-SIGIR 2002*.
 - Variable byte codes
- V. N. Anh and A. Moffat. 2005. Inverted Index Compression Using Word-Aligned Binary Codes. *Information Retrieval 8*: 151–166.
 - Word aligned codes

Resources

- IIR 3, MG 4.2
- Efficient spell retrieval:
 - K. Kukich. Techniques for automatically correcting words in text. ACM Computing Surveys 24(4), Dec 1992.
 - J. Zobel and P. Dart. Finding approximate matches in large lexicons. Software - practice and experience 25(3), March 1995.
<http://citeseer.ist.psu.edu/zobel95finding.html>
 - Mikael Tillenius: Efficient Generation and Ranking of Spelling Error Corrections. Master's thesis at Sweden's Royal Institute of Technology.
<http://citeseer.ist.psu.edu/179155.html>

Questions?

